GUÍA AMPLIADA DE JAVASCRIPT

Condicionales · Bucles · Objetos · Funciones · Arrays · Clases

APUNTE ORIGINAL	AMPLIACIÓN	EJEMPLO
Texto que estaba en tus TXT. Se conserva sin reescribir.	Explicación nueva añadida para entenderlo mejor.	Código nuevo, separado visualmente del apunte.

Esta versión reorganiza el material de los dos TXT para que avance desde los fundamentos hasta arrays y termine con clases, manteniendo los apuntes originales y separando visualmente todo lo que se añade después.

ÍNDICE NAVEGABLE

Haz clic en cualquier apartado del índice o usa los marcadores del panel lateral del PDF para saltar directamente al tema.

1.5.1. Condicionales en JavaScript 4

1.5.2. Bucles básicos en JavaScript 6

1.5.3. Bucles avanzados con objetos y arrays 8

1.5.4. Control de flujo con break y continue 10

2.1. Objeto String 12

2.2. Objeto Number 15

2.3. Objeto Math 18

2.4. Objeto Date 21

2.5. Objeto Boolean 24

2.7. Set 26

2.8. Map 29

2.9. Symbol 32

3.1.1. Funciones (I): Fundamentos 34

3.1.2. Funciones (II): Uso avanzado 36

3.1.3. Funciones (III): Parámetros y closures 38

Material adicional de funciones - TXT 1 40

3.2.1. Array (I): Fundamentos 50

3.2.2. Array (II): Iteración y transformación 52

3.2.3. Array (III): Métodos de búsqueda y ordenación 54

3.2.4. Array (IV): Manipulación avanzada y buenas prácticas 57

3.3.1. Clases y objetos (I): fundamentos 61

3.3.2. Clases y objetos (II): herencia y encapsulación 64

Repaso final y mapa mental 67

Cómo leer esta guía

APUNTE ORIGINAL	AMPLIACIÓN	EJEMPLO
Texto que estaba en tus TXT. Se conserva sin reescribir.	Explicación nueva añadida para entenderlo mejor.	Código nuevo, separado visualmente del apunte.

• 1. Lee primero el bloque original. Está arriba de cada tema y conserva la explicación que venía en los TXT.

• 2. Baja después a la ampliación. Ahí encontrarás el significado práctico, ideas para memorizar y un ejemplo nuevo.

• 3. Los ejemplos de código tienen otra caja y otra tipografía. Así no se confunden con el texto explicado.

• 4. El recorrido está ordenado. Se empieza por condicionales y bucles, después objetos, funciones, arrays y, al final, clases.

NOTA SOBRE LOS DOS TXT

El primer TXT también contiene material de funciones. Para no perder nada de lo que entregaste, ese contenido se conserva más adelante dentro del capítulo de funciones como material adicional del TXT 1.

1.5.1. Condicionales en JavaScript

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

1.5.1. Condicionales en JavaScript

En programación, los condicionales nos permiten tomar decisiones: ejecutar un bloque de código u otro en función de si una condición se cumple.

Sentencia if

La sentencia if evalúa una condición.

Si es verdadera (true), se ejecuta el bloque de código asociado.

```
let edad = 20;
```

```
if (edad >= 18) {
  console.log("Eres mayor de edad");
}
```

if...else

Con else podemos ejecutar un bloque alternativo si la condición no se cumple.

```
let edad = 16;
```

```
if (edad >= 18) {
  console.log("Eres mayor de edad");
} else {
  console.log("Eres menor de edad");
}
```

if...else if...else

Sirve para evaluar múltiples condiciones en orden. El primer bloque que sea verdadero detiene la evaluación.

```
let nota = 7;
```

```
if (nota >= 9) {
  console.log("Sobresaliente");
} else if (nota >= 7) {
  console.log("Notable");
} else if (nota >= 5) {
  console.log("Aprobado");
} else {
  console.log("Suspenso");
}
```

Condiciones compuestas

Podemos combinar condiciones con operadores lógicos:

&& (AND): ambas deben cumplirse

|| (OR): al menos una debe cumplirse

! (NOT): niega el resultado

```
let usuario = "Laura";
let logueado = true;
```

```
if (usuario === "Laura" && logueado) {
  console.log("Acceso permitido");
}
```

Sentencia switch

El switch permite comparar un valor con múltiples casos. Es útil cuando se evalúa una sola variable con varios posibles valores.

```
let dia = "martes";
```

```
switch (dia) {
  case "lunes":
    console.log("Inicio de semana");
    break;
  case "martes":
  case "miércoles":
  case "jueves":
    console.log("Semana en curso");
    break;
  case "viernes":
    console.log("¡Viernes!");
    break;
```

default:

```
    console.log("Fin de semana");
}
```

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Un condicional no "hace" una operación por sí mismo: decide qué parte del programa debe ejecutarse. La condición se evalúa y produce un valor booleano. Con if/else se elige entre caminos; con else if se encadenan varias comprobaciones y con switch se agrupan casos concretos.

IDEAS CLAVE

- Usa === y !== cuando quieras evitar conversiones de tipo inesperadas.
- Piensa en el orden de las condiciones: la primera condición verdadera puede impedir que se evalúen las siguientes.
- Cuando una condición empieza a ser muy larga, separar la lógica en variables booleanas suele hacer el código más fácil de leer.

EJEMPLO NUEVO

```
const edad = 20;
const tieneEntrada = true;

if (edad >= 18 && tieneEntrada) {
  console.log("Puede entrar");
} else {
  console.log("No puede entrar");
}
```

1.5.2. Bucles básicos en JavaScript

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

1.5.2. Bucles básicos en JavaScript

En programación muchas veces necesitamos repetir tareas.

Los bucles permiten ejecutar un bloque de código varias veces, mientras se cumpla una condición.

Bucle for

El bucle for es el más común cuando sabemos cuántas veces queremos repetir algo.

Su estructura tiene tres partes: inicialización, condición y actualización.

```
for (let i = 1; i <= 5; i++) {
  console.log(`Iteración número ${i}`);
}
```

En este ejemplo:

```
let i = 1: inicializa la variable de control.
```

i <= 5: condición que se evalúa en cada vuelta.

i++: incrementa la variable en cada iteración.

Bucle while

El bucle while repite un bloque de código mientras la condición sea verdadera. Es útil cuando no sabemos de antemano cuántas iteraciones habrá.

```
let contador = 0;
```

```
while (contador < 3) {
  console.log(`Contador: ${contador}`);
  contador++;
}
```

Bucle do...while

El bucle do...while es similar al while, pero garantiza que el código se ejecute al menos una vez, aunque la condición sea falsa desde el principio.

```
let numero = 5;
```

```
do {
  console.log(`El número es ${numero}`);
  numero++;
} while (numero < 3);
```

En este caso, aunque la condición es falsa desde el inicio, el bloque se ejecuta una vez.

Diferencia clave

while: primero evalúa la condición y luego ejecuta.

do...while: ejecuta primero y evalúa después.

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Los bucles repiten una tarea. La diferencia principal está en cómo controlan la repetición: for suele ser cómodo cuando conoces el contador o el número de iteraciones; while es útil cuando la repetición depende de una condición; do...while ejecuta el cuerpo al menos una vez.

IDEAS CLAVE

- Comprueba siempre qué hace que el bucle termine.
- Actualiza las variables de control dentro del bucle cuando corresponda.
- Un while sin cambio de estado puede convertirse en un bucle infinito.

EJEMPLO NUEVO

```
let suma = 0;

for (let i = 1; i <= 5; i++) {
  suma += i;
}

console.log(suma); // 15
```

1.5.3. Bucles avanzados con objetos y arrays

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

1.5.3. Bucles avanzados con objetos y arrays

Además de los bucles tradicionales (for, while, do...while), JavaScript ofrece estructuras especiales para trabajar con arrays y objetos.

Estos bucles hacen el código más legible y moderno.

Bucle for...in

El bucle for...in se utiliza para recorrer las propiedades enumerables de un objeto.

```
const persona = {
```

nombre: "Laura",

nif: 12345678A,

ciudad: "Zamora"

```
};
```

```
for (let clave in persona) {
  console.log(`${clave}: ${persona[clave]}`);
}
```

Salida:

nombre: Laura

edad: 12345678A

ciudad: Zamora

Cuidado con for...in

for...in no está pensado para recorrer arrays, ya que puede incluir propiedades añadidas al prototipo.

Bucle for...of

El bucle for...of se introdujo en ES6 y se usa para recorrer elementos de arrays, strings, Maps, Sets y otros iterables.

```
const frutas = ["manzana", "pera", "plátano"];
```

```
for (let fruta of frutas) {
  console.log(fruta);
}
```

Salida:

manzana

pera

plátano

Métodos modernos para arrays

Los arrays en JavaScript tienen métodos especiales que reemplazan muchos usos de bucles tradicionales que veremos en lecciones posteriores.

forEach

Ejecuta una función para cada elemento.

```
const numeros = [1, 2, 3];
```

```
numeros.forEach(n => console.log(n * 2));
// 2, 4, 6
```

map

Crea un nuevo array aplicando una función a cada elemento.

```
const dobles = numeros.map(n => n * 2);
console.log(dobles); // [2, 4, 6]
```

filter

Crea un nuevo array con los elementos que cumplen una condición.

```
const pares = numeros.filter(n => n % 2 === 0);
console.log(pares); // [2]
```

reduce

Reduce los elementos a un único valor (por ejemplo, una suma).

```
const suma = numeros.reduce((acum, n) => acum + n, 0);
console.log(suma); // 6
```

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Aquí aparecen formas más expresivas de recorrer datos. for...of está pensado para recorrer valores de iterables como arrays o strings; for...in recorre nombres de propiedades y resulta especialmente útil con objetos.

IDEAS CLAVE

- En un array, for...of suele expresar mejor que quieres trabajar con cada valor.
- En objetos, for...in recorre claves, no directamente los valores.
- map, filter y reduce permiten describir una transformación sin gestionar manualmente un contador.

EJEMPLO NUEVO

```
const notas = [5, 7, 8, 10];

const aprobadas = notas.filter(nota => nota >= 5);
const conExtra = aprobadas.map(nota => nota + 1);

console.log(conExtra);
```

1.5.4. Control de flujo con break y continue

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

1.5.4 Control de flujo con break y continue

En algunos casos necesitamos alterar el flujo normal de un bucle.

Para ello, JavaScript ofrece dos instrucciones clave: break y continue.

La instrucción break

```
break se utiliza para salir de un bucle inmediatamente, incluso si la condición aún no se ha cumplido.
```

```
for (let i = 1; i <= 10; i++) {
  if (i === 5) {
    console.log("¡Se interrumpe el bucle en i = 5!");
    break;
  }
  console.log(i);
}
```

Salida:

1

2

3

4

¡Se interrumpe el bucle en i = 5!

En este caso, el bucle se detiene en cuanto i vale 5.

La instrucción continue

```
continue se utiliza para saltar la iteración actual y pasar directamente a la siguiente.
```

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) {
    console.log("Saltamos el número 3");
    continue;
  }
  console.log(i);
}
```

Salida:

1

2

Saltamos el número 3

4

5

En este ejemplo, cuando i es 3, no se ejecuta el console.log(i) y se pasa a la siguiente vuelta.

Usos prácticos

```
break: útil cuando buscamos un elemento y podemos detener el bucle al encontrarlo.
continue: útil para ignorar casos específicos dentro de un bucle.
const numeros = [1, 2, -5, 3, 0, 4];
```

```
for (let n of numeros) {
  if (n < 0) {
    console.log("Número negativo encontrado, detenemos el bucle");
    break;
  }
  if (n === 0) {
    console.log("Ignoramos el 0");
    continue;
  }
  console.log(`Número válido: ${n}`);
}
```

Salida:

Número válido: 1

Número válido: 2

Número negativo encontrado, detenemos el bucle

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

break y continue no son condiciones: son instrucciones que alteran un bucle que ya está funcionando. break termina el bucle completo; continue salta la iteración actual y pasa a la siguiente.

IDEAS CLAVE

- Usa break cuando ya no tiene sentido seguir recorriendo.
- Usa continue para ignorar casos concretos sin abandonar el bucle.
- Demasiados saltos dentro de un bucle pueden dificultar la lectura; conviene que la intención sea evidente.

EJEMPLO NUEVO

```
for (let i = 1; i <= 6; i++) {
  if (i === 2) continue;
  if (i === 5) break;

  console.log(i);
}
```

2.1. Objeto String

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

2.1 Objeto String

El objeto String en JavaScript se utiliza para trabajar con cadenas de texto. Aunque los strings son un tipo primitivo, cuando usas métodos sobre ellos el motor crea temporalmente un objeto wrapper para darte acceso a sus propiedades y métodos.

Inmutabilidad de las cadenas

Las cadenas son inmutables: ningún método modifica la cadena original. Todos los métodos devuelven una nueva cadena.

Creación de cadenas

Se pueden crear de dos formas aunque se recomienda siempre usar la primera: la forma literal.

```
// Forma literal (recomendada)
let texto1 = "Hola mundo";
```

```
// Constructor: crea un objeto String ( evita usarlo)
let texto2 = new String("Hola mundo");
```

No uses new String() para crear cadenas

```
new String("hola") crea un objeto, no una cadena primitiva.
```

Puede romper comparaciones estrictas: "hola" === new String("hola") es false.

typeof "hola" es "string", pero typeof new String("hola") es "object". Recomendación: usa siempre literales con comillas ("...", '...') o plantillas (`...`).

Propiedades principales

```
// .length → longitud de la cadena (número de caracteres)
let mensaje = "JavaScript";
console.log(mensaje.length); // 10
```

Métodos más utilizados de String

Cambio de mayúsculas y minúsculas

```
// .toUpperCase() → convierte a MAYÚSCULAS
console.log("hola".toUpperCase()); // "HOLA"
```

```
// .toLowerCase() → convierte a minúsculas
console.log("HOLA".toLowerCase()); // "hola"
```

Obtener partes de una cadena

```
let cadena = "JavaScript";
```

```
// .substring(inicio, finExclusivo) → sin índices negativos
console.log(cadena.substring(0, 4)); // "Java"
```

```
// .slice(inicio, finExclusivo) → acepta índices negativos
console.log(cadena.slice(4));   // "Script"
console.log(cadena.slice(-6));  // "Script"
```

Búsqueda de texto

```
let texto = "Aprendiendo JavaScript";
```

```
// .indexOf(subcadena) → índice de la primera aparición o -1
console.log(texto.indexOf("Java")); // 12
```

```
// .includes(subcadena) → true si contiene
console.log(texto.includes("Script")); // true
```

```
// .startsWith(subcadena) → true si empieza por...
console.log(texto.startsWith("Apr")); // true
```

```
// .endsWith(subcadena) → true si termina con...
console.log(texto.endsWith("pt")); // true
```

```
// .charAt(indice) → caracter en la posición dada
console.log(texto.charAt(12)); // "J"
```

Reemplazo de texto

```
let saludo = "Hola Laura";
```

```
// .replace(buscar, reemplazo) → reemplaza la primera coincidencia
console.log(saludo.replace("Laura", "Mundo")); // "Hola Mundo"
```

```
// .replaceAll(buscar, reemplazo) → reemplaza todas (ES2021)
console.log("lala".replaceAll("la", "na")); // "nana"
```

```
// Con expresiones regulares globales (alternativa a replaceAll)
console.log("lala".replace(/la/g, "na")); // "nana"
```

Eliminación de espacios

```
let espacio = "   Hola   ";
```

```
// .trim() → elimina espacios al inicio y al final
console.log(espacio.trim()); // "Hola"
```

```
// .trimStart() → solo al inicio
console.log(espacio.trimStart()); // "Hola   "
```

```
// .trimEnd() → solo al final
console.log(espacio.trimEnd()); // "   Hola"
```

Repetición y división

```
// .repeat(n) → repite la cadena n veces
console.log("ha".repeat(3)); // "hahaha"
```

```
// .split(separador) → divide y devuelve un array
const colores = "rojo verde azul".split(" "); // ["rojo", "verde", "azul"]
```

```
// (Array) .join(separador) → une un array en una cadena
console.log(colores.join(" - ")); // "rojo - verde - azul"
```

Plantillas literales (Template literals)

Desde ES6, las comillas invertidas (`) permiten interpolar variables/expresiones y escribir múltiples líneas.

```
let nombre = "Laura";
let edad = 25;
```

```
// Interpolación con ${expresión}
console.log(`Me llamo ${nombre} y el año que viene tendré ${edad + 1} años.`);
```

```
// Multilínea sin caracteres especiales
let poema = `Esto es
```

una cadena

```
multilínea.`;
console.log(poema);
```

Cuándo usar plantillas

Cuando necesites interpolación (${...}).

Para cadenas multilínea sin \n.

Para construir cadenas complejas de forma legible.

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

String representa texto y ofrece métodos para consultar, buscar y transformar cadenas. Un detalle importante es que los strings son inmutables: métodos como toUpperCase() o trim() devuelven un nuevo string en lugar de cambiar el original.

IDEAS CLAVE

- length indica cuántos caracteres tiene el string.
- includes, startsWith y endsWith sirven para preguntas frecuentes sobre texto.
- Las template literals con backticks facilitan insertar variables dentro de una cadena.

EJEMPLO NUEVO

```
const nombre = "  Laura  ";

const limpio = nombre.trim();
const mensaje = `Hola, ${limpio}`;

console.log(mensaje); // Hola, Laura
```

2.2. Objeto Number

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

2.2 Objeto Number

El objeto Number en JavaScript se utiliza para representar y trabajar con valores numéricos. JavaScript no distingue entre enteros y decimales: todos los números se representan internamente como números de coma flotante de 64 bits (doble precisión, IEEE 754).

Enteros y decimales en JavaScript

Aunque escribas 10 o 10.5, ambos se representan con el mismo tipo: number. Esto puede generar resultados inesperados con operaciones de decimales (por ejemplo, 0.1 + 0.2 0.3 exactamente).

Creación de números

```
// Forma literal (recomendada)
let a = 42;
let b = 3.14;
```

```
// Constructor Number ( evita usarlo, crea un objeto Number)
let c = new Number(42);
```

```
console.log(typeof a); // "number"
console.log(typeof c); // "object"
```

Propiedades estáticas de Number

```
// Número máximo representable
console.log(Number.MAX_VALUE);
```

```
// Número mínimo positivo representable
console.log(Number.MIN_VALUE);
```

```
// Valor especial "no es un número"
console.log(Number.NaN);
```

```
// Valores infinitos
console.log(Number.POSITIVE_INFINITY);
console.log(Number.NEGATIVE_INFINITY);
```

Métodos de instancia más comunes

Estos métodos se aplican a valores numéricos.

```
let x = 123.456;
```

```
// .toFixed(n) → fija el número de decimales
console.log(x.toFixed(2)); // "123.46"
```

```
// .toPrecision(n) → total de cifras significativas
console.log(x.toPrecision(4)); // "123.5"
```

```
// .toString(base) → convierte a string en otra base (por defecto, base 10)
console.log((255).toString(16)); // "ff"
```

Métodos estáticos de Number

```
// Number.isInteger(n) → true si es un entero
console.log(Number.isInteger(42)); // true
console.log(Number.isInteger(3.14)); // false
```

```
// Number.isNaN(valor) → comprueba si es NaN real (mejor que isNaN global)
console.log(Number.isNaN("hola")); // false
console.log(Number.isNaN(NaN));    // true
```

```
// Number.isFinite(valor) → true si es un número finito
console.log(Number.isFinite(100));    // true
console.log(Number.isFinite(Infinity)); // false
```

```
// Number.parseInt(cadena) → convierte a entero
console.log(Number.parseInt("42px")); // 42
```

```
// Number.parseFloat(cadena) → convierte a número decimal
console.log(Number.parseFloat("3.14 metros")); // 3.14
```

Cuidado con parseInt() sin base

Si no indicas la base numérica (radix), parseInt() puede interpretarlo de forma distinta en navegadores antiguos. Siempre es recomendable usar: js parseInt("1010", 2); // 10 en base 2

BigInt: números muy grandes

En ES2020 se introdujo BigInt para trabajar con enteros más grandes que Number.MAX_SAFE_INTEGER.

```
// Sintaxis: añadir "n" al final o usar BigInt()
let grande = 9007199254740991n; // Mayor que MAX_SAFE_INTEGER
let otro = BigInt("1234567890123456789012345678890");
```

```
console.log(typeof grande); // "bigint"
```

```
// Operaciones con BigInt (solo entre BigInt)
console.log(grande + 2n);
console.log(otro * 10n);
```

BigInt y Number no se mezclan

No puedes sumar un Number normal y un BigInt directamente. Debes convertir uno al otro de forma explícita.

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Number es el tipo numérico habitual de JavaScript. Además de números normales, conviene reconocer valores especiales como NaN e Infinity, porque afectan a comparaciones y cálculos.

IDEAS CLAVE

- NaN significa "Not a Number", pero su tipo sigue siendo number.
- Number.isFinite() es útil para comprobar que un valor es un número finito.
- Para cantidades enteras extremadamente grandes existe BigInt, que es un tipo distinto.

EJEMPLO NUEVO

```
const texto = "12.5";
const numero = Number(texto);

if (Number.isFinite(numero)) {
  console.log(numero + 1);
}
```

2.3. Objeto Math

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

2.3 Objeto Math

El objeto Math en JavaScript proporciona constantes y funciones matemáticas. A diferencia de Number, no se instancia: se accede directamente como Math.función() o Math.CONSTANTE.

Constantes y propiedades de Math

```
// Math.PI → valor de π
console.log(Math.PI); // 3.141592653589793
```

```
// Math.E → base del logaritmo natural
console.log(Math.E); // 2.718281828459045
```

```
// Math.SQRT2 → raíz cuadrada de 2
console.log(Math.SQRT2); // 1.4142135623730951
```

```
// Math.LN2 → logaritmo natural de 2
console.log(Math.LN2); // 0.6931471805599453
```

Uso de Math

Todas las propiedades y métodos de Math son estáticos. No puedes hacer new Math().

Redondeo y valores absolutos

```
// Math.round() → redondea al entero más cercano
console.log(Math.round(4.6)); // 5
console.log(Math.round(4.4)); // 4
```

```
// Math.floor() → redondea hacia abajo
console.log(Math.floor(4.9)); // 4
```

```
// Math.ceil() → redondea hacia arriba
console.log(Math.ceil(4.1)); // 5
```

```
// Math.trunc() → elimina los decimales
console.log(Math.trunc(4.9)); // 4
```

```
// Math.abs() → valor absoluto
console.log(Math.abs(-7)); // 7
```

Máximos, mínimos y potencias

```
// Math.max() → devuelve el mayor valor
console.log(Math.max(2, 8, 5)); // 8
```

```
// Math.min() → devuelve el menor valor
console.log(Math.min(2, 8, 5)); // 2
```

```
// Math.pow(base, exponente) → potencia
console.log(Math.pow(2, 3)); // 8
```

```
// Equivalente moderno con el operador **
console.log(2 ** 3); // 8
```

```
// Math.sqrt(x) → raíz cuadrada
console.log(Math.sqrt(16)); // 4
```

Aleatoriedad

```
// Math.random() → número aleatorio entre 0 y 1 (sin incluir el 1)
console.log(Math.random());
```

```
// Número aleatorio entre 0 y 10
console.log(Math.random() * 10);
```

```
// Número aleatorio entero entre 1 y 6 (como un dado)
console.log(Math.floor(Math.random() * 6) + 1);
```

Math.random() no es seguro

Math.random() no es criptográficamente seguro. Para seguridad (tokens, contraseñas) usa la API crypto.

Funciones trigonométricas

```
// Math.sin(x), Math.cos(x), Math.tan(x) → ángulo en radianes
console.log(Math.sin(Math.PI / 2)); // 1
console.log(Math.cos(0));           // 1
console.log(Math.tan(Math.PI / 4)); // 1
```

```
// Inversas: asin, acos, atan
console.log(Math.asin(1)); // 1.5707963267948966 (π/2)
console.log(Math.acos(1)); // 0
console.log(Math.atan(1)); // 0.7853981633974483 (π/4)
```

Logaritmos y exponenciales

```
// Logaritmo natural
console.log(Math.log(Math.E)); // 1
```

```
// Logaritmo base 10
console.log(Math.log10(100)); // 2
```

```
// Logaritmo base 2
console.log(Math.log2(8)); // 3
```

```
// Exponencial (e^x)
console.log(Math.exp(1)); // 2.718281828459045
```

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Math concentra funciones matemáticas ya preparadas. No se crea con new Math(): se usan directamente propiedades y métodos como Math.round(), Math.floor(), Math.ceil(), Math.max() y Math.random().

IDEAS CLAVE

- round redondea al entero más cercano; floor baja y ceil sube.
- Math.random() genera un valor pseudoaleatorio entre 0 y 1.
- Para generar enteros en un intervalo hay que controlar bien el límite superior.

EJEMPLO NUEVO

```
const minimo = 1;
const maximo = 6;

const dado = Math.floor(
  Math.random() * (maximo - minimo + 1)
) + minimo;

console.log(dado);
```

2.4. Objeto Date

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

2.4 Objeto Date

El objeto Date en JavaScript permite trabajar con fechas y horas. Se utiliza para obtener la fecha actual, crear fechas específicas o manipularlas (sumar días, calcular diferencias, etc.).

Representación interna de las fechas

Internamente, una fecha en JavaScript se guarda como el número de milisegundos desde el 1 de enero de 1970 a las 00:00:00 UTC (Epoch time).

Creación de fechas

```
// Fecha y hora actuales
let ahora = new Date();
console.log(ahora);
```

```
// Crear a partir de una fecha en string
let navidad = new Date("2025-12-25");
console.log(navidad);
```

```
// Crear con valores numéricos (año, mes, día, hora, minuto, segundo, milisegundo)
//  Importante: los meses empiezan en 0 (0 = enero, 11 = diciembre)
let ejemplo = new Date(2025, 11, 31, 23, 59, 59);
console.log(ejemplo);
```

Métodos para obtener partes de la fecha

```
let fecha = new Date("2025-09-16T10:30:00");
```

```
// Día del mes
console.log(fecha.getDate()); // 16
```

```
// Día de la semana (0 = domingo, 6 = sábado)
console.log(fecha.getDay()); // 2 (martes)
```

```
// Mes (0 = enero, 11 = diciembre)
console.log(fecha.getMonth()); // 8 (septiembre)
```

```
// Año
console.log(fecha.getFullYear()); // 2025
```

```
// Horas, minutos y segundos
console.log(fecha.getHours());   // 10
console.log(fecha.getMinutes()); // 30
console.log(fecha.getSeconds()); // 0
```

Cuidado con los meses

getMonth() devuelve valores de 0 a 11. Ejemplo: enero = 0, septiembre = 8, diciembre = 11.

Métodos para establecer valores

```
let fecha = new Date("2025-01-01");
```

```
// Cambiar el año
fecha.setFullYear(2030);
console.log(fecha);
```

```
// Cambiar mes (recuerda: enero = 0)
```

fecha.setMonth(6); // julio

```
console.log(fecha);
```

```
// Cambiar día
fecha.setDate(15);
console.log(fecha);
```

Formatos de salida

```
let hoy = new Date();
```

```
// Formato legible local
console.log(hoy.toLocaleDateString()); // ej. "16/9/2025"
```

```
// Fecha y hora completas locales
console.log(hoy.toLocaleString()); // ej. "16/9/2025, 10:45:00"
```

```
// Solo la hora
console.log(hoy.toLocaleTimeString()); // ej. "10:45:00"
```

```
// ISO 8601 (formato estándar internacional)
console.log(hoy.toISOString()); // "2025-09-16T08:45:00.000Z"
```

Operaciones con fechas

```
let inicio = new Date("2025-09-01");
let fin = new Date("2025-09-16");
```

```
// Diferencia en milisegundos
let diferencia = fin - inicio;
console.log(diferencia); // 1296000000
```

```
// Convertir a días
let dias = diferencia / (1000 * 60 * 60 * 24);
console.log(dias); // 15
```

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Date representa un instante temporal y permite trabajar con fechas y horas. En JavaScript conviene distinguir entre crear una fecha, leer sus partes y formatearla para mostrarla.

IDEAS CLAVE

- Los meses de getMonth() se numeran desde 0: enero es 0 y diciembre es 11.
- Una fecha es fácil de intercambiar usando formatos como ISO 8601.
- Las fechas con zonas horarias pueden provocar diferencias si se mezclan métodos locales y UTC.

EJEMPLO NUEVO

```
const hoy = new Date();

console.log(hoy.getFullYear());
console.log(hoy.getMonth() + 1);
console.log(hoy.toISOString());
```

2.5. Objeto Boolean

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

2.5 Objeto Boolean

El objeto Boolean en JavaScript representa valores lógicos: verdadero (true) o falso (false). Se utiliza en estructuras de control como condicionales (if, while, etc.) y en comparaciones.

Creación de valores booleanos

```
// Forma literal (recomendada)
let esMayor = true;
let esMenor = false;
```

```
// Constructor Boolean ( evita usarlo, crea un objeto)
let b = new Boolean(true);
```

```
console.log(typeof esMayor); // "boolean"
console.log(typeof b);       // "object"
```

No uses new Boolean()

Si usas new Boolean(false), el resultado es un objeto que siempre se evalúa como true en contextos lógicos. Ejemplo: js if (new Boolean(false)) { console.log("Esto se ejecuta "); }

Conversión a booleano

Cualquier valor en JavaScript puede convertirse a un booleano.

```
// Boolean() convierte de forma explícita
console.log(Boolean(1));       // true
console.log(Boolean(0));       // false
console.log(Boolean("hola"));  // true
console.log(Boolean(""));      // false
```

Valores truthy y falsy

Falsy: valores que se convierten a false son false, 0, -0, 0n, "", null, undefined, NaN

Truthy: todos los demás valores (incluidos arrays y objetos vacíos).

```
console.log(Boolean([]));  // true
console.log(Boolean({}));  // true
```

Evaluación de truthy/falsy

Este mecanismo es muy usado en condicionales:

```
`   let nombre = "";
    if (nombre) {
    console.log("Hay un nombre");
    } else {
    console.log("Cadena vacía → se evalúa como false");
    }
```

`

Operadores lógicos principales

```
// AND lógico (&&) → true si ambos son verdaderos
console.log(true && false); // false
```

```
// OR lógico (||) → true si al menos uno es verdadero
console.log(true || false); // true
```

```
// NOT lógico (!) → invierte el valor
console.log(!true); // false
```

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Boolean representa una decisión lógica: true o false. La conversión booleana es especialmente importante porque JavaScript considera algunos valores "truthy" y otros "falsy" dentro de condiciones.

IDEAS CLAVE

- false, 0, "", null, undefined y NaN son ejemplos de valores falsy.
- Todo lo demás es truthy en muchas situaciones de control.
- Boolean(valor) permite ver explícitamente cómo se convierte un valor a booleano.

EJEMPLO NUEVO

```
const nombre = "";

if (nombre) {
  console.log("Hay nombre");
} else {
  console.log("No hay nombre");
}
```

2.7. Set

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

2.7 Set: Conjuntos de valores únicos

El objeto Set permite almacenar valores únicos (sin duplicados) y mantener el orden de inserción. Es ideal para eliminar duplicados o comprobar pertenencia de forma eficiente.

Valores únicos y comparación

Set considera dos valores iguales siguiendo la semántica SameValueZero: NaN se considera igual a NaN, y +0 y -0 se consideran el mismo valor.

¿Qué es un Set?

Un Set es una colección de elementos sin índices y sin claves, donde cada valor aparece como mucho una vez. Se recorre en el orden en el que se insertaron los elementos.

Crear un Set

Literal y desde arrays

```
// Set vacío
const s1 = new Set();
```

```
// Desde un array (los duplicados se eliminan)
const s2 = new Set([1, 2, 2, 3, 3, 4]); // Set {1, 2, 3, 4}
```

Eliminar duplicados rápidamente

```
const numeros = [1, 2, 2, 3, 4, 4, 5];
const unicos = [...new Set(numeros)]; // [1, 2, 3, 4, 5]
```

Métodos principales

Añadir y comprobar

```
const letras = new Set();
```

letras.add("a"); // añade "a"

letras.add("b"); // añade "b"

letras.add("a"); // duplicado: no tiene efecto

letras.has("a"); // true

letras.has("z"); // false

Eliminar elementos

letras.delete("b"); // elimina "b" si existe

letras.delete("x"); // false si no existía

Tamaño y limpieza

letras.size; // número de elementos del Set

letras.clear(); // elimina todos los elementos

Recorrer un Set

Con for...of y forEach

```
const colores = new Set(["rojo", "verde", "azul"]);
```

```
for (const color of colores) {
  console.log(color);
}
```

```
colores.forEach(c => console.log(c));
```

Convertir entre Set y Array

De Array a Set y vuelta

```
const arr = ["a", "b", "b", "c"];
const set = new Set(arr);      // Set {"a","b","c"}
const backToArray = [...set];  // ["a","b","c"]
```

Operaciones típicas de conjuntos

Aunque Set no trae métodos nativos para unión/intersección/diferencia, se pueden construir fácilmente.

Unión

```
const A = new Set([1, 2, 3]);
const B = new Set([3, 4, 5]);
const union = new Set([...A, ...B]);          // Set {1,2,3,4,5}
```

Intersección

```
const interseccion = new Set([...A].filter(x => B.has(x))); // Set {3}
```

Diferencia

```
const diferencia = new Set([...A].filter(x => !B.has(x)));  // Set {1,2}
```

Casos de uso frecuentes

Quitar duplicados de una lista

```
const emails = ["a@x.com", "b@x.com", "a@x.com"];
const unicos = [...new Set(emails)]; // ["a@x.com","b@x.com"]
```

Comprobación rápida de pertenencia

```
const palabrasProhibidas = new Set(["spam", "fake", "scam"]);
```

palabrasProhibidas.has("spam"); // true

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Set es una colección en la que cada valor aparece una sola vez. Es especialmente práctico para eliminar duplicados o preguntar si un elemento está presente sin gestionar manualmente índices.

IDEAS CLAVE

- new Set(array) es una forma habitual de deduplicar un array.
- size indica cuántos elementos únicos hay.
- has() comprueba pertenencia y add()/delete() cambian la colección.

EJEMPLO NUEVO

```
const numeros = [1, 2, 2, 3, 3, 3];

const unicos = [...new Set(numeros)];

console.log(unicos); // [1, 2, 3]
```

2.8. Map

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

2.8 Map: Colecciones clave–valor

El objeto Map permite almacenar pares clave–valor, donde las claves pueden ser de cualquier tipo (objetos, funciones, arrays, valores primitivos). A diferencia de los objetos normales ({}), un Map mantiene el orden de inserción y ofrece métodos específicos para su gestión.

¿Por qué usar Map en lugar de objetos literales?

En un objeto literal, las claves siempre se convierten a cadenas.

En un Map, las claves pueden ser cualquier valor, incluso objetos o funciones.

Además, Map ofrece el método .size y una API más clara para trabajar con colecciones dinámicas.

Crear un Map

Map vacío y con valores iniciales

```
// Map vacío
const m1 = new Map();
```

```
// Map con pares clave-valor iniciales
const m2 = new Map([
  ["nombre", "Laura"],
  ["edad", 30],
  [true, "activo"]
]);
```

Usando objetos o funciones como clave

```
const claveObjeto = { id: 1 };
const claveFuncion = function() {};
```

```
const m3 = new Map();
m3.set(claveObjeto, "soy un objeto");
m3.set(claveFuncion, "soy una función");
```

Métodos principales

Añadir y obtener

```
const persona = new Map();
```

```
persona.set("nombre", "Laura");
persona.set("edad", 28);
```

```
console.log(persona.get("nombre")); // "Laura"
console.log(persona.get("edad"));   // 28
```

Comprobar y eliminar

persona.has("nombre"); // true

persona.delete("edad"); // elimina la clave "edad"

Tamaño y limpieza

persona.size; // número de elementos

persona.clear(); // elimina todo el contenido

Recorrer un Map

Con for...of

```
const usuario = new Map([
  ["nombre", "Laura"],
  ["rol", "admin"],
  ["activo", true]
]);
```

```
for (const [clave, valor] of usuario) {
  console.log(clave, valor);
}
```

Con forEach

```
usuario.forEach((valor, clave) => {
  console.log(`${clave}: ${valor}`);
});
```

Obtener listas de claves, valores y entradas

```
[...usuario.keys()];   // ["nombre", "rol", "activo"]
[...usuario.values()]; // ["Laura", "admin", true]
[...usuario.entries()];// [["nombre","Laura"],["rol","admin"],["activo",true]]
```

Diferencias clave entre Map y objetos literales

Mantiene el orden de inserción.

Las claves pueden ser cualquier tipo de dato.

Proporciona la propiedad .size en lugar de Object.keys(obj).length.

Iterar sobre un Map es más directo y claro.

Casos de uso frecuentes

Contador de elementos

```
const palabras = ["uno", "dos", "uno", "tres", "dos", "uno"];
const contador = new Map();
```

```
for (const palabra of palabras) {
  contador.set(palabra, (contador.get(palabra) || 0) + 1);
}
```

```
console.log(contador); // Map { "uno" => 3, "dos" => 2, "tres" => 1 }
```

Asociar objetos sin modificar su estructura

```
const usuario1 = { id: 1 };
const usuario2 = { id: 2 };
```

```
const roles = new Map();
roles.set(usuario1, "admin");
roles.set(usuario2, "editor");
```

```
console.log(roles.get(usuario1)); // "admin"
```

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Map es una colección de pares clave-valor. A diferencia de un objeto normal, las claves pueden ser de cualquier tipo, incluso objetos o funciones, y tiene una API específica para trabajar con ellas.

IDEAS CLAVE

- set() añade o reemplaza una clave; get() recupera su valor.
- has() comprueba si existe una clave y delete() la elimina.
- size indica cuántos pares clave-valor contiene el Map.

EJEMPLO NUEVO

```
const visitas = new Map();

visitas.set("Ana", 3);
visitas.set("Luis", 5);

console.log(visitas.get("Luis")); // 5
```

2.9. Symbol

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

2.9 Symbol: Identificadores únicos

El tipo de dato Symbol fue introducido en ES6 y sirve para crear identificadores únicos e inmutables. Se utiliza principalmente para evitar colisiones de nombres en propiedades de objetos y para definir claves privadas o semiprivadas.

Dato importante

Cada vez que llamas a Symbol(), se genera un valor nuevo y único, aunque uses la misma descripción.

Crear un Symbol

Con descripción opcional

```
// Crear dos símbolos diferentes aunque tengan la misma descripción
const s1 = Symbol("id");
const s2 = Symbol("id");
```

```
console.log(s1 === s2); // false
```

La descripción ("id") es solo un texto útil para depuración, no afecta a la identidad del símbolo.

Usar Symbol como clave en objetos

```
const ID = Symbol("id");
```

```
const usuario = {
```

nombre: "Laura",

```
  [ID]: 12345
};
```

```
console.log(usuario.nombre); // "Laura"
console.log(usuario[ID]);    // 12345
```

Las propiedades definidas con Symbol no aparecen en bucles normales como for...in o Object.keys().

Recuperar propiedades Symbol

Aunque no aparecen en listados normales, puedes acceder a ellas con métodos especiales:

```
const simbolos = Object.getOwnPropertySymbols(usuario);
console.log(simbolos); // [ Symbol(id) ]
```

Symbol global (Symbol.for)

Si necesitas reutilizar un símbolo, puedes guardarlo en el registro global con Symbol.for.

```
const global1 = Symbol.for("claveGlobal");
const global2 = Symbol.for("claveGlobal");
```

```
console.log(global1 === global2); // true
```

Con Symbol.keyFor() puedes recuperar la clave de un símbolo global:

```
console.log(Symbol.keyFor(global1)); // "claveGlobal"
```

Symbol y las APIs de JavaScript

Existen símbolos bien conocidos que el lenguaje usa internamente, como:

Symbol.iterator: define cómo se itera sobre un objeto.

Symbol.toPrimitive: personaliza cómo se convierte un objeto a primitivo.

Symbol.toStringTag: cambia la etiqueta de Object.prototype.toString.

Ejemplo con Symbol.iterator:

```
const numeros = [1, 2, 3];
const iterador = numeros[Symbol.iterator]();
```

```
console.log(iterador.next()); // { value: 1, done: false }
console.log(iterador.next()); // { value: 2, done: false }
```

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Symbol sirve para crear identificadores únicos. Dos símbolos con la misma descripción siguen siendo distintos, por lo que resultan útiles como claves de propiedades cuando quieres evitar colisiones de nombres.

IDEAS CLAVE

- Symbol("x") y Symbol("x") no son iguales.
- La descripción es informativa; la identidad la aporta el símbolo.
- Los símbolos pueden usarse como propiedades de un objeto.

EJEMPLO NUEVO

```
const idInterno = Symbol("id");

const usuario = {
  nombre: "Ana",
  [idInterno]: 42
};

console.log(usuario[idInterno]);
```

3.1.1. Funciones (I): Fundamentos

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

3.1.1. Funciones (I): Fundamentos

Las funciones son bloques de código que permiten organizar, reutilizar y estructurar mejor un programa.

Declaración de funciones tradicionales

Las funciones tradicionales se definen con la palabra clave function.

Se pueden declarar con nombre (funciones nombradas) o sin nombre (funciones anónimas, que veremos más adelante).

```
// Función nombrada
function saludar(nombre) {
  return `Hola, ${nombre}`;
}
```

```
console.log(saludar("Laura")); // Hola, Laura
```

Funciones flecha (arrow functions)

Las funciones flecha se introdujeron en ES6 y ofrecen una forma más corta y expresiva de escribir funciones. Además, heredan el this del contexto donde se crean.

Sintaxis básica

```
const saludar = () => {
  console.log("Hola mundo");
};
```

saludar(); // Hola mundo

Con un solo argumento

```
const saludar = nombre => {
  console.log(`Hola, ${nombre}`);
};
```

saludar("Laura"); // Hola, Laura

Con varios argumentos

```
const sumar = (a, b) => {
  return a + b;
};
```

```
console.log(sumar(3, 4)); // 7
```

Retorno implícito

```
const cuadrado = x => x * x;
```

```
console.log(cuadrado(5)); // 25
```

Sin argumentos

```
const obtenerHora = () => new Date().toLocaleTimeString();
```

```
console.log(obtenerHora()); // Ejemplo: "10:45:12"
```

Funciones flecha como callbacks

Las funciones flecha se usan mucho como callbacks en métodos de arrays (map, filter, reduce), porque hacen el código más claro.

```
const numeros = [1, 2, 3, 4, 5];
```

```
// Con función tradicional
const dobles1 = numeros.map(function(n) {
  return n * 2;
});
```

```
// Con función flecha
const dobles2 = numeros.map(n => n * 2);
```

```
console.log(dobles2); // [2, 4, 6, 8, 10]
```

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Una función agrupa instrucciones que pueden reutilizarse. Al declararla, describes una operación; al invocarla, ejecutas esa operación. Los parámetros reciben datos de entrada y return permite producir un resultado para quien llama a la función.

IDEAS CLAVE

- Elige nombres de función que expliquen la acción: calcularTotal, validarEdad, obtenerUsuario.
- Una función puede tener cero, uno o varios parámetros.
- return termina la ejecución de la función y devuelve un valor.

EJEMPLO NUEVO

```
function calcularTotal(precio, cantidad) {
  return precio * cantidad;
}

const total = calcularTotal(12, 3);
console.log(total); // 36
```

3.1.2. Funciones (II): Uso avanzado

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

3.1.2. Funciones (II): Uso avanzado

En esta lección vamos a profundizar en otros usos comunes de las funciones en JavaScript:

funciones anónimas, funciones de callback y parámetros por defecto.

Funciones anónimas

Una función anónima es aquella que no tiene nombre propio.

Se suelen usar asignándolas a una variable o pasándolas directamente como argumento a otra función.

```
// Asignada a una variable
const cuadrado = function(x) {
  return x * x;
};
```

```
console.log(cuadrado(4)); // 16
```

Funciones de callback

Un callback es una función que se pasa como argumento a otra y que se ejecuta después de que esta la llame. Son muy comunes en JavaScript, especialmente en asincronía y en métodos de arrays.

```
function procesarUsuario(nombre, callback) {
  console.log(`Procesando usuario: ${nombre}`);
  callback();
}
```

```
// Pasamos una función anónima como callback
procesarUsuario("Laura", function() {
  console.log("Callback ejecutado ");
});
```

```
// O una función flecha
procesarUsuario("Laura", () => {
  console.log("Callback con función flecha ");
});
```

Parámetros por defecto

Los parámetros por defecto permiten dar un valor inicial a un argumento de la función si no se proporciona al llamarla. Esto evita tener que comprobar manualmente si el parámetro existe.

```
function saludar(nombre = "invitado") {
  console.log(`Hola, ${nombre}`);
}
```

saludar(); // Hola, invitado

saludar("Laura"); // Hola, Laura

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

En JavaScript las funciones son valores: se pueden guardar en variables, pasar como argumentos y devolver desde otras funciones. De ahí sale gran parte del uso de callbacks y del estilo funcional del lenguaje.

IDEAS CLAVE

- Una función callback se pasa a otra función para que esta la ejecute.
- Las funciones flecha son una sintaxis compacta muy usada para callbacks.
- El contexto de this tiene reglas propias; las funciones flecha no crean su propio this.

EJEMPLO NUEVO

```
function procesar(valor, callback) {
  return callback(valor);
}

const resultado = procesar(5, numero => numero * 2);

console.log(resultado); // 10
```

3.1.3. Funciones (III): Parámetros y closures

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

3.1.3. Funciones (III): Parámetros y closures

En esta lección aprenderás a manejar funciones que aceptan un número variable de argumentos y a comprender el concepto de closure, clave en JavaScript.

El objeto arguments

Antes de ES6, cuando queríamos que una función aceptara un número indefinido de argumentos, se usaba el objeto especial arguments.

Este objeto se genera automáticamente dentro de cualquier función tradicional (no funciona en funciones flecha).

```
function sumar() {
  let total = 0;
  for (let i = 0; i < arguments.length; i++) {
    total += arguments[i];
  }
  return total;
}
```

```
console.log(sumar(1, 2, 3)); // 6
```

Problemas de arguments

No es un array real, sino un objeto parecido a array (array-like).

No funciona en funciones flecha.

Hace el código menos claro y menos moderno.

Parámetros rest (...)

Con ES6 llegaron los parámetros rest, que sustituyen a arguments de una forma más clara. Los valores extra se agrupan en un array real, mucho más cómodo de usar.

```
function sumar(...numeros) {
  return numeros.reduce((a, b) => a + b, 0);
}
```

```
console.log(sumar(1, 2, 3));     // 6
console.log(sumar(4, 5, 6, 7));  // 22
```

Ventajas frente a arguments

Genera arrays reales (con métodos como .map, .filter, .reduce).

Funciona en cualquier tipo de función, incluidas las flecha.

Hace el código más legible y moderno.

Closures (cierres)

Un closure ocurre cuando una función recuerda el entorno en el que fue creada, incluso después de que ese entorno haya terminado.

Sirve para encapsular datos y crear variables privadas.

```
function crearContador() {
  let contador = 0;
```

```
  return function() {
    contador++;
    return contador;
  };
}
```

```
const contar = crearContador();
```

```
console.log(contar()); // 1
console.log(contar()); // 2
console.log(contar()); // 3
```

En este ejemplo, la variable contador se mantiene en memoria gracias al closure, aunque la función crearContador ya haya terminado su ejecución.

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Los parámetros rest permiten recoger un número variable de argumentos en un array. Un closure aparece cuando una función interna conserva acceso a variables de su entorno aunque la función externa ya haya terminado.

IDEAS CLAVE

- ...args debe ser el último parámetro.
- El closure permite mantener estado privado sin exponer directamente la variable.
- Es una base importante para callbacks, fábricas de funciones y encapsulación.

EJEMPLO NUEVO

```
function crearContador() {
  let contador = 0;

  return () => {
    contador++;
    return contador;
  };
}

const siguiente = crearContador();

console.log(siguiente()); // 1
console.log(siguiente()); // 2
```

Material adicional de funciones presente en el TXT 1

Este bloque también forma parte de los archivos entregados. Se mantiene separado para que puedas identificar su procedencia sin mezclarlo con la ampliación didáctica.

CONTENIDO ORIGINAL - SIN MODIFICAR (MATERIAL PRESENTE EN EL TXT 1)

3.1.1. Funciones (I): Fundamentos

Las funciones son bloques de código que permiten organizar, reutilizar y estructurar mejor un programa.

Declaración de funciones tradicionales

Las funciones tradicionales se definen con la palabra clave function.

Se pueden declarar con nombre (funciones nombradas) o sin nombre (funciones anónimas, que veremos más adelante).

```
// Función nombrada
function saludar(nombre) {
  return `Hola, ${nombre}`;
}
```

```
console.log(saludar("Laura")); // Hola, Laura
```

Funciones flecha (arrow functions)

Las funciones flecha se introdujeron en ES6 y ofrecen una forma más corta y expresiva de escribir funciones. Además, heredan el this del contexto donde se crean.

Sintaxis básica

```
const saludar = () => {
  console.log("Hola mundo");
};
```

saludar(); // Hola mundo

Con un solo argumento

```
const saludar = nombre => {
  console.log(`Hola, ${nombre}`);
};
```

saludar("Laura"); // Hola, Laura

Con varios argumentos

```
const sumar = (a, b) => {
  return a + b;
};
```

```
console.log(sumar(3, 4)); // 7
```

Retorno implícito

```
const cuadrado = x => x * x;
```

```
console.log(cuadrado(5)); // 25
```

Sin argumentos

```
const obtenerHora = () => new Date().toLocaleTimeString();
```

```
console.log(obtenerHora()); // Ejemplo: "10:45:12"
```

Funciones flecha como callbacks

Las funciones flecha se usan mucho como callbacks en métodos de arrays (map, filter, reduce), porque hacen el código más claro.

```
const numeros = [1, 2, 3, 4, 5];
```

```
// Con función tradicional
const dobles1 = numeros.map(function(n) {
  return n * 2;
});
```

```
// Con función flecha
const dobles2 = numeros.map(n => n * 2);
```

```
console.log(dobles2); // [2, 4, 6, 8, 10]//3.1.2. Funciones (II): Uso avanzado
```

En esta lección vamos a profundizar en otros usos comunes de las funciones en JavaScript:

funciones anónimas, funciones de callback y parámetros por defecto.

Funciones anónimas

Una función anónima es aquella que no tiene nombre propio.

Se suelen usar asignándolas a una variable o pasándolas directamente como argumento a otra función.

```
// Asignada a una variable
const cuadrado = function(x) {
  return x * x;
};
```

```
console.log(cuadrado(4)); // 16
```

Funciones de callback

Un callback es una función que se pasa como argumento a otra y que se ejecuta después de que esta la llame. Son muy comunes en JavaScript, especialmente en asincronía y en métodos de arrays.

```
function procesarUsuario(nombre, callback) {
  console.log(`Procesando usuario: ${nombre}`);
  callback();
}
```

```
// Pasamos una función anónima como callback
procesarUsuario("Laura", function() {
  console.log("Callback ejecutado ");
});
```

```
// O una función flecha
procesarUsuario("Laura", () => {
  console.log("Callback con función flecha ");
});
```

Parámetros por defecto

Los parámetros por defecto permiten dar un valor inicial a un argumento de la función si no se proporciona al llamarla. Esto evita tener que comprobar manualmente si el parámetro existe.

```
function saludar(nombre = "invitado") {
  console.log(`Hola, ${nombre}`);
}
```

saludar(); // Hola, invitado

saludar("Laura"); // Hola, Laura//3.1.3. Funciones (III): Parámetros y closures

En esta lección aprenderás a manejar funciones que aceptan un número variable de argumentos

y a comprender el concepto de closure, clave en JavaScript.

El objeto arguments

Antes de ES6, cuando queríamos que una función aceptara un número indefinido de argumentos,

se usaba el objeto especial arguments.

Este objeto se genera automáticamente dentro de cualquier función tradicional (no funciona en funciones flecha).

```
function sumar() {
  let total = 0;
  for (let i = 0; i < arguments.length; i++) {
    total += arguments[i];
  }
  return total;
}
```

```
console.log(sumar(1, 2, 3)); // 6
```

Problemas de arguments

No es un array real, sino un objeto parecido a array (array-like).

No funciona en funciones flecha.

Hace el código menos claro y menos moderno.

Parámetros rest (...)

Con ES6 llegaron los parámetros rest, que sustituyen a arguments de una forma más clara. Los valores extra se agrupan en un array real, mucho más cómodo de usar.

```
function sumar(...numeros) {
  return numeros.reduce((a, b) => a + b, 0);
}
```

```
console.log(sumar(1, 2, 3));     // 6
console.log(sumar(4, 5, 6, 7));  // 22
```

Ventajas frente a arguments

Genera arrays reales (con métodos como .map, .filter, .reduce).

Funciona en cualquier tipo de función, incluidas las flecha.

Hace el código más legible y moderno.

Closures (cierres)

Un closure ocurre cuando una función recuerda el entorno en el que fue creada, incluso después de que ese entorno haya terminado.

Sirve para encapsular datos y crear variables privadas.

```
function crearContador() {
  let contador = 0;
```

```
  return function() {
    contador++;
    return contador;
  };
}
```

```
const contar = crearContador();
```

```
console.log(contar()); // 1
console.log(contar()); // 2
console.log(contar()); // 3
```

En este ejemplo, la variable contador se mantiene en memoria gracias al closure, aunque la función crearContador ya haya terminado su ejecución.

3.2.2 Array (II): Iteración y transformación

Además de almacenar datos, los arrays en JavaScript ofrecen muchos métodos para **recorrerlos** y **transformarlos**. Estos métodos son fundamentales para trabajar de forma moderna y clara con colecciones de datos.

Recorrer un array con bucles tradicionales

Puedes recorrer un array con los bucles que ya conoces (`for`, `for...of`, etc.).

```

```
let frutas = ["manzana", "plátano", "naranja"];
```

```
// Con un for clásico
for (let i = 0; i < frutas.length; i++) {
  console.log(frutas[i]);
}
```

```
// Con for...of
for (let fruta of frutas) {
  console.log(fruta);
}
```
```

```

## Otros métodos de iteración y transformación

### forEach()

El método `.forEach()` ejecuta una función para **cada elemento** del array.

```

```
let numeros = [1, 2, 3, 4];
```

```
numeros.forEach((n) => {
  console.log(n * 2);
});
// 2, 4, 6, 8
```

```

### map()

El método `.map()` crea un **nuevo array** aplicando una función a cada elemento.

```

```
let numeros = [1, 2, 3, 4];
```

```
// Crear un nuevo array con los números al cuadrado
let cuadrados = numeros.map((n) => n * n);
```

```
console.log(cuadrados); // [1, 4, 9, 16]
```

```

### filter()

El método `.filter()` devuelve un **nuevo array** con los elementos que cumplen una condición.

```

```
let numeros = [1, 2, 3, 4, 5, 6];
```

```
// Filtrar los pares
let pares = numeros.filter((n) => n % 2 === 0);
```

```
console.log(pares); // [2, 4, 6]
```

```

### reduce()

El método `.reduce()` acumula los valores del array en un único resultado.

```

```
let numeros = [1, 2, 3, 4];
```

```
// Sumar todos los números
let suma = numeros.reduce((acumulador, actual) => acumulador + actual, 0);
```

```
console.log(suma); // 10//.2.3 Array (III): Métodos de búsqueda y ordenación
```

Además de recorrer y transformar arrays, JavaScript proporciona métodos para buscar elementos y ordenar datos de manera eficiente.

Métodos de búsqueda

indexOf() y lastIndexOf()

indexOf(valor) devuelve la primera posición en la que aparece un valor (o -1 si no existe).

lastIndexOf(valor) devuelve la última posición.

```
let frutas = ["manzana", "pera", "naranja", "pera"];
```

```
console.log(frutas.indexOf("pera"));     // 1
console.log(frutas.lastIndexOf("pera")); // 3
console.log(frutas.indexOf("kiwi"));     // -1
```

includes()

Comprueba si un array contiene un valor.

```
let numeros = [10, 20, 30];
```

```
console.log(numeros.includes(20)); // true
console.log(numeros.includes(40)); // false
```

find() y findIndex()

find() devuelve el primer elemento que cumple una condición.

findIndex() devuelve la posición de ese elemento.

```
let numeros = [5, 12, 8, 130, 44];
```

```
console.log(numeros.find(n => n > 10));      // 12
console.log(numeros.findIndex(n => n > 10)); // 1
```

some() y every()

some() devuelve true si algún elemento cumple la condición.

every() devuelve true si todos los elementos cumplen la condición.

```
let numeros = [2, 4, 6, 8];
```

```
console.log(numeros.some(n => n > 5));         // true
console.log(numeros.every(n => n % 2 === 0));  // true
```

Métodos de ordenación

sort()

Ordena los elementos de un array. Por defecto, ordena como strings, lo que puede dar resultados inesperados.

```
let letras = ["d", "a", "c", "b"];
console.log(letras.sort()); // ["a", "b", "c", "d"]
```

```
let numeros = [10, 2, 30, 1];
// ¡Ojo! Los ordena como strings
console.log(numeros.sort()); // [1, 10, 2, 30]
```

```
// Orden numérico correcto
console.log(numeros.sort((a, b) => a - b)); // [1, 2, 10, 30]
```

Cuidado con sort()

```
Si no proporcionas una función de comparación, sort() convierte los elementos a texto y los
ordena alfabéticamente. Por eso, al ordenar números, es recomendable siempre dar una función
(a, b) => a - b.
```

reverse()

Invierte el orden de los elementos de un array.

```
let numeros = [1, 2, 3, 4, 5];
console.log(numeros.reverse()); // [5, 4, 3, 2, 1]
```

Inciso sobre sort()

El método sort() convierte los elementos a cadenas de texto antes de ordenarlos, lo que provoca que los números no siempre se ordenen como esperarías.

Ejemplo con números

```
let numeros = [2, 100, 15, 7];
```

```
// Ordena como si fueran strings
console.log(numeros.sort());
// [100, 15, 2, 7]
```

```
// Orden correcto con función de comparación
console.log(numeros.sort((a, b) => a - b));
// [2, 7, 15, 100]
```

Por qué ocurre esto

sort() sin función compara los valores como texto, siguiendo el orden Unicode. Por eso "100" se coloca antes que "2", igual que en un diccionario "c" va antes que "ca".

Ejemplo con objetos

Si tenemos un array de objetos, también necesitamos una función de comparación para ordenar por alguna propiedad.

```
let personas = [
  { nombre: "Laura", edad: 28 },
  { nombre: "Ana", edad: 22 },
  { nombre: "Pedro", edad: 35 }
];
```

```
// Ordenar por edad (de menor a mayor)
console.log(personas.sort((a, b) => a.edad - b.edad));
/*
[
  { nombre: "Ana", edad: 22 },
  { nombre: "Laura", edad: 28 },
  { nombre: "Pedro", edad: 35 }
]*/
```

```
// Ordenar por nombre (alfabéticamente)
console.log(personas.sort((a, b) => a.nombre.localeCompare(b.nombre)));
/*
[
  { nombre: "Ana", edad: 22 },
  { nombre: "Laura", edad: 28 },
  { nombre: "Pedro", edad: 35 }
]
*///const nums = [10, 20, 30, 40];
```

```
// slice(inicio, finExcl) → devuelve una porción sin modificar el original
const parte = nums.slice(1, 3);     // [20, 30]
```

```
// concat(...arrs) → une arrays, no muta
const unidos = [1, 2].concat([3, 4]); // [1, 2, 3, 4]
```

```
// spread (...) → copia superficial o unión rápida
const copia = [...nums];              // [10, 20, 30, 40]
const unidos2 = [...[1, 2], ...[3, 4]]; // [1, 2, 3, 4]
```

Reordenación no destructiva (ES2023)

```
const a = [3, 1, 2];
```

```
// toSorted(cmp?) → como sort() pero NO muta
const ordenado = a.toSorted((x, y) => x - y); // [1, 2, 3]; a sigue [3,1,2]
```

```
// toReversed() → como reverse() pero NO muta
const invertido = a.toReversed(); // [2,1,3]; a sigue [3,1,2]
```

```
// toSpliced(idx, del, ...ins) → como splice() pero NO muta
const sinPrimero = a.toSpliced(0, 1);      // [1,2]
const conInsercion = a.toSpliced(1, 0, 9); // [3,9,1,2]
```

```
// with(index, value) → copia cambiando un solo índice (no muta)
const b = [0, 1, 2];
const b2 = b.with(1, 99); // [0, 99, 2]; b sigue [0,1,2]
```

Acceso cómodo

```
const letras = ["a", "b", "c", "d"];
```

```
// at(i) → acepta índices negativos
```

letras.at(-1); // "d"

letras.at(-2); // "c"

¿Por qué preferir métodos no mutables?

Al no modificar el array original, evitas efectos colaterales y facilitas el razonamiento y el depurado del código, especialmente en UIs reactivas.

Métodos que sí mutan el array (úsalos con cuidado)

Estos cambian el contenido del array original.

Inserción/eliminación flexible

```
const frutas = ["manzana", "pera", "plátano"];
```

```
// splice(idx, deleteCount, ...items) → elimina/insertar/reemplaza en el sitio (muta)
```

frutas.splice(1, 1); // elimina "pera" ["manzana","plátano"]

frutas.splice(1, 0, "kiwi"); // inserta en 1 ["manzana","kiwi","plátano"]

frutas.splice(2, 1, "uva"); // reemplaza en 2 ["manzana","kiwi","uva"]

Ordenación y reverso en sitio

```
const nums = [10, 2, 30, 1];
```

```
// sort(cmp?) → ordena como strings si no pasas comparador (muta)
nums.sort((a, b) => a - b); // [1,2,10,30]
```

```
// reverse() → invierte en sitio (muta)
```

nums.reverse(); // [30,10,2,1]

Cuidado con sort() por defecto

```
Sin comparador, sort() convierte a texto y puede ordenar números de forma inesperada (p. ej.,
10 antes que 2). Usa siempre (a,b) => a - b para orden numérico.
```

Relleno y copia en bloque

```
const arr = [1, 2, 3, 4];
```

```
// fill(valor, inicio=0, fin=arr.length) → rellena en sitio
```

arr.fill(0, 1, 3); // [1,0,0,4]

```
// copyWithin(dest, inicio, finExcl) → copia un segmento dentro del propio array
const z = [10, 20, 30, 40];
```

z.copyWithin(1, 2); // copia [30,40] sobre idx1 [10,30,40,40]

Aplanado y mapeo

```
// flat(profundidad=1) → aplana arrays anidados
[1, [2, 3], [4, [5, 6]]].flat();   // [1, 2, 3, 4, [5, 6]]
[1, [2, 3], [4, [5, 6]]].flat(2);  // [1, 2, 3, 4, 5, 6]
```

```
// flatMap(fn) → mapea y aplana un nivel
[1, 2, 3].flatMap(n => [n, n * 2]); // [1,2, 2,4, 3,6]
```

Creación y verificación de arrays

```
// Array.isArray(valor) → comprueba si es un array
```

Array.isArray([1, 2, 3]); // true

```
// Array.of(...valores) → crea array con los argumentos
```

Array.of(3); // [3]

Array.of(1, 2, 3); // [1,2,3]

```
// Array.from(iterable, mapFn?) → crea desde iterable o array-like, con mapeo opcional
```

Array.from("hola"); // ["h","o","l","a"]

```
Array.from({ length: 4 }, (_, i) => i * 2); // [0,2,4,6]
```

Patrones útiles (inmutables)

Insertar, reemplazar y eliminar sin mutar

```
const arr = [10, 20, 30, 40];
```

```
// Insertar 99 en la posición 2 (sin mutar)
const insertado = arr.toSpliced(2, 0, 99); // [10,20,99,30,40]
```

```
// Reemplazar el valor en la posición 1 por 77 (sin mutar)
const reemplazado = arr.with(1, 77);       // [10,77,30,40]
```

```
// Eliminar el elemento en la posición 0 (sin mutar)
const sinPrimero = arr.toSpliced(0, 1);    // [20,30,40]
```

Ordenar sin mutar

```
const datos = [3, 1, 2];
```

```
// Orden ascendente sin tocar el original
const asc = datos.toSorted((a, b) => a - b); // [1,2,3]
```

Clonar de forma segura (superficial)

```
const original = [{ id: 1 }, { id: 2 }];
```

```
// Copia superficial con spread
const copia = [...original];        // los objetos internos se comparten
```

```
```

3.2.1. Array (I): Fundamentos

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

3.2.1 Array (I): Fundamentos

Los arrays en JavaScript son estructuras de datos que permiten almacenar varios valores en una sola variable. Son muy útiles cuando queremos trabajar con listas de elementos (números, cadenas, objetos...).

¿Qué es un array?

Un array es una lista ordenada de elementos que se indexan comenzando desde 0. Cada valor dentro del array se llama elemento, y se accede a él mediante su índice.

```
// Declaración de un array
let frutas = ["manzana", "plátano", "naranja"];
```

```
// Acceder al primer elemento
console.log(frutas[0]); // "manzana"
```

```
// Acceder al segundo elemento
console.log(frutas[1]); // "plátano"
```

Longitud de un array

La propiedad .length devuelve el número total de elementos en el array.

```
let frutas = ["manzana", "plátano", "naranja"];
console.log(frutas.length); // 3
```

Arrays multidimensionales

Un array puede contener otros arrays en su interior, formando arrays multidimensionales. Esto es útil, por ejemplo, para representar tablas.

```
let tabla = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];
```

```
console.log(tabla[0][1]); // 2 (primera fila, segunda columna)
```

Métodos básicos para modificar arrays

Los arrays incluyen métodos muy usados para añadir o eliminar elementos.

```
let numeros = [1, 2, 3];
```

```
// Añadir al final
numeros.push(4);
console.log(numeros); // [1, 2, 3, 4]
```

```
// Eliminar el último
numeros.pop();
console.log(numeros); // [1, 2, 3]
```

```
// Añadir al principio
numeros.unshift(0);
console.log(numeros); // [0, 1, 2, 3]
```

```
// Eliminar el primero
numeros.shift();
console.log(numeros); // [1, 2, 3]
```

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Un array es una colección ordenada de valores. Cada posición tiene un índice que empieza en 0. Los arrays pueden contener valores de distintos tipos, aunque en código real suele ser más sencillo trabajar con colecciones homogéneas.

IDEAS CLAVE

- length cuenta posiciones, no necesariamente elementos "válidos" cuando hay huecos.
- push/pop trabajan normalmente por el final; shift/unshift por el principio.
- Los arrays pueden contener otros arrays y formar estructuras multidimensionales.

EJEMPLO NUEVO

```
const tareas = ["estudiar", "practicar"];

tareas.push("repasar");

console.log(tareas[0]);     // estudiar
console.log(tareas.length); // 3
```

3.2.2. Array (II): Iteración y transformación

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

3.2.2 Array (II): Iteración y transformación

Además de almacenar datos, los arrays en JavaScript ofrecen muchos métodos para recorrerlos y transformarlos. Estos métodos son fundamentales para trabajar de forma moderna y clara con colecciones de datos.

Recorrer un array con bucles tradicionales

Puedes recorrer un array con los bucles que ya conoces (for, for...of, etc.).

```
let frutas = ["manzana", "plátano", "naranja"];
```

```
// Con un for clásico
for (let i = 0; i < frutas.length; i++) {
  console.log(frutas[i]);
}
```

```
// Con for...of
for (let fruta of frutas) {
  console.log(fruta);
}
```

Otros métodos de iteración y transformación

forEach()

El método .forEach() ejecuta una función para cada elemento del array.

```
let numeros = [1, 2, 3, 4];
```

```
numeros.forEach((n) => {
  console.log(n * 2);
});
// 2, 4, 6, 8
```

map()

El método .map() crea un nuevo array aplicando una función a cada elemento.

```
let numeros = [1, 2, 3, 4];
```

```
// Crear un nuevo array con los números al cuadrado
let cuadrados = numeros.map((n) => n * n);
```

```
console.log(cuadrados); // [1, 4, 9, 16]
```

filter()

El método .filter() devuelve un nuevo array con los elementos que cumplen una condición.

```
let numeros = [1, 2, 3, 4, 5, 6];
```

```
// Filtrar los pares
let pares = numeros.filter((n) => n % 2 === 0);
```

```
console.log(pares); // [2, 4, 6]
```

reduce()

El método .reduce() acumula los valores del array en un único resultado.

```
let numeros = [1, 2, 3, 4];
```

```
// Sumar todos los números
let suma = numeros.reduce((acumulador, actual) => acumulador + actual, 0);
```

```
console.log(suma); // 10
```

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Estos métodos permiten recorrer y transformar arrays sin escribir siempre un for manual. map cambia la representación de cada elemento, filter conserva algunos, forEach ejecuta una acción y reduce acumula un resultado.

IDEAS CLAVE

- map devuelve un nuevo array.
- filter devuelve un nuevo array con los elementos que cumplen una condición.
- reduce no está limitado a sumar: puede construir objetos, contadores o cualquier acumulación.

EJEMPLO NUEVO

```
const precios = [10, 20, 30];

const conIVA = precios.map(precio => precio * 1.21);
const caros = conIVA.filter(precio => precio > 20);

console.log(caros);
```

3.2.3. Array (III): Métodos de búsqueda y ordenación

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

3.2.3 Array (III): Métodos de búsqueda y ordenación

Además de recorrer y transformar arrays, JavaScript proporciona métodos para buscar elementos y ordenar datos de manera eficiente.

Métodos de búsqueda

indexOf() y lastIndexOf()

indexOf(valor) devuelve la primera posición en la que aparece un valor (o -1 si no existe).

lastIndexOf(valor) devuelve la última posición.

```
let frutas = ["manzana", "pera", "naranja", "pera"];
```

```
console.log(frutas.indexOf("pera"));     // 1
console.log(frutas.lastIndexOf("pera")); // 3
console.log(frutas.indexOf("kiwi"));     // -1
```

includes()

Comprueba si un array contiene un valor.

```
let numeros = [10, 20, 30];
```

```
console.log(numeros.includes(20)); // true
console.log(numeros.includes(40)); // false
```

find() y findIndex()

find() devuelve el primer elemento que cumple una condición.

findIndex() devuelve la posición de ese elemento.

```
let numeros = [5, 12, 8, 130, 44];
```

```
console.log(numeros.find(n => n > 10));      // 12
console.log(numeros.findIndex(n => n > 10)); // 1
```

some() y every()

some() devuelve true si algún elemento cumple la condición.

every() devuelve true si todos los elementos cumplen la condición.

```
let numeros = [2, 4, 6, 8];
```

```
console.log(numeros.some(n => n > 5));         // true
console.log(numeros.every(n => n % 2 === 0));  // true
```

Métodos de ordenación

sort()

Ordena los elementos de un array. Por defecto, ordena como strings, lo que puede dar resultados inesperados.

```
let letras = ["d", "a", "c", "b"];
console.log(letras.sort()); // ["a", "b", "c", "d"]
```

```
let numeros = [10, 2, 30, 1];
// ¡Ojo! Los ordena como strings
console.log(numeros.sort()); // [1, 10, 2, 30]
```

```
// Orden numérico correcto
console.log(numeros.sort((a, b) => a - b)); // [1, 2, 10, 30]
```

Cuidado con sort()

```
Si no proporcionas una función de comparación, sort() convierte los elementos a texto y los
ordena alfabéticamente. Por eso, al ordenar números, es recomendable siempre dar una función
(a, b) => a - b.
```

reverse()

Invierte el orden de los elementos de un array.

```
let numeros = [1, 2, 3, 4, 5];
console.log(numeros.reverse()); // [5, 4, 3, 2, 1]
```

Inciso sobre sort()

El método sort() convierte los elementos a cadenas de texto antes de ordenarlos, lo que provoca que los números no siempre se ordenen como esperarías.

Ejemplo con números

```
let numeros = [2, 100, 15, 7];
```

```
// Ordena como si fueran strings
console.log(numeros.sort());
// [100, 15, 2, 7]
```

```
// Orden correcto con función de comparación
console.log(numeros.sort((a, b) => a - b));
// [2, 7, 15, 100]
```

Por qué ocurre esto

sort() sin función compara los valores como texto, siguiendo el orden Unicode. Por eso "100" se coloca antes que "2", igual que en un diccionario "c" va antes que "ca".

Ejemplo con objetos

Si tenemos un array de objetos, también necesitamos una función de comparación para ordenar por alguna propiedad.

```
let personas = [
  { nombre: "Laura", edad: 28 },
  { nombre: "Ana", edad: 22 },
  { nombre: "Pedro", edad: 35 }
];
```

```
// Ordenar por edad (de menor a mayor)
console.log(personas.sort((a, b) => a.edad - b.edad));
/*
[
  { nombre: "Ana", edad: 22 },
  { nombre: "Laura", edad: 28 },
  { nombre: "Pedro", edad: 35 }
]*/
```

```
// Ordenar por nombre (alfabéticamente)
console.log(personas.sort((a, b) => a.nombre.localeCompare(b.nombre)));
/*
[
  { nombre: "Ana", edad: 22 },
  { nombre: "Laura", edad: 28 },
  { nombre: "Pedro", edad: 35 }
]
*/
```

Consejo

Cuando trabajes con arrays de objetos, piensa siempre qué propiedad quieres usar para ordenar y define una función de comparación adecuada.

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Buscar y ordenar son operaciones distintas. find devuelve el primer elemento que cumple una condición; indexOf e includes trabajan especialmente bien con valores concretos. sort reordena el array original, salvo que utilices una alternativa que cree una copia.

IDEAS CLAVE

- some responde si al menos un elemento cumple; every si todos cumplen.
- sort trata los elementos como texto si no le pasas comparador numérico.
- Cuando solo quieres consultar sin modificar, conviene usar métodos que no muten el array.

EJEMPLO NUEVO

```
const numeros = [10, 2, 30];

const ordenados = [...numeros].sort((a, b) => a - b);

console.log(ordenados); // [2, 10, 30]
console.log(numeros);   // [10, 2, 30]
```

3.2.4. Array (IV): Manipulación avanzada y buenas prácticas

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

3.2.4 Array (IV): Manipulación avanzada y buenas prácticas

En esta lección veremos cómo copiar, reordenar, insertar, eliminar y aplanar arrays, con especial atención a los métodos no mutables introducidos recientemente y a patrones recomendados.

Métodos que no mutan el array original (recomendados)

Estos devuelven nuevos arrays y dejan intacto el original. Son ideales cuando quieres mantener inmutabilidad (por ejemplo, en interfaces reactivas).

Copias y cortes (no destructivos)

```
const nums = [10, 20, 30, 40];
```

```
// slice(inicio, finExcl) → devuelve una porción sin modificar el original
const parte = nums.slice(1, 3);     // [20, 30]
```

```
// concat(...arrs) → une arrays, no muta
const unidos = [1, 2].concat([3, 4]); // [1, 2, 3, 4]
```

```
// spread (...) → copia superficial o unión rápida
const copia = [...nums];              // [10, 20, 30, 40]
const unidos2 = [...[1, 2], ...[3, 4]]; // [1, 2, 3, 4]
```

Reordenación no destructiva (ES2023)

```
const a = [3, 1, 2];
```

```
// toSorted(cmp?) → como sort() pero NO muta
const ordenado = a.toSorted((x, y) => x - y); // [1, 2, 3]; a sigue [3,1,2]
```

```
// toReversed() → como reverse() pero NO muta
const invertido = a.toReversed(); // [2,1,3]; a sigue [3,1,2]
```

```
// toSpliced(idx, del, ...ins) → como splice() pero NO muta
const sinPrimero = a.toSpliced(0, 1);      // [1,2]
const conInsercion = a.toSpliced(1, 0, 9); // [3,9,1,2]
```

```
// with(index, value) → copia cambiando un solo índice (no muta)
const b = [0, 1, 2];
const b2 = b.with(1, 99); // [0, 99, 2]; b sigue [0,1,2]
```

Acceso cómodo

```
const letras = ["a", "b", "c", "d"];
```

```
// at(i) → acepta índices negativos
```

letras.at(-1); // "d"

letras.at(-2); // "c"

¿Por qué preferir métodos no mutables?

Al no modificar el array original, evitas efectos colaterales y facilitas el razonamiento y el depurado del código, especialmente en UIs reactivas.

Métodos que sí mutan el array (úsalos con cuidado)

Estos cambian el contenido del array original.

Inserción/eliminación flexible

```
const frutas = ["manzana", "pera", "plátano"];
```

```
// splice(idx, deleteCount, ...items) → elimina/insertar/reemplaza en el sitio (muta)
```

frutas.splice(1, 1); // elimina "pera" ["manzana","plátano"]

frutas.splice(1, 0, "kiwi"); // inserta en 1 ["manzana","kiwi","plátano"]

frutas.splice(2, 1, "uva"); // reemplaza en 2 ["manzana","kiwi","uva"]

Ordenación y reverso en sitio

```
const nums = [10, 2, 30, 1];
```

```
// sort(cmp?) → ordena como strings si no pasas comparador (muta)
nums.sort((a, b) => a - b); // [1,2,10,30]
```

```
// reverse() → invierte en sitio (muta)
```

nums.reverse(); // [30,10,2,1]

Cuidado con sort() por defecto

```
Sin comparador, sort() convierte a texto y puede ordenar números de forma inesperada (p. ej.,
10 antes que 2). Usa siempre (a,b) => a - b para orden numérico.
```

Relleno y copia en bloque

```
const arr = [1, 2, 3, 4];
```

```
// fill(valor, inicio=0, fin=arr.length) → rellena en sitio
```

arr.fill(0, 1, 3); // [1,0,0,4]

```
// copyWithin(dest, inicio, finExcl) → copia un segmento dentro del propio array
const z = [10, 20, 30, 40];
```

z.copyWithin(1, 2); // copia [30,40] sobre idx1 [10,30,40,40]

Aplanado y mapeo

```
// flat(profundidad=1) → aplana arrays anidados
[1, [2, 3], [4, [5, 6]]].flat();   // [1, 2, 3, 4, [5, 6]]
[1, [2, 3], [4, [5, 6]]].flat(2);  // [1, 2, 3, 4, 5, 6]
```

```
// flatMap(fn) → mapea y aplana un nivel
[1, 2, 3].flatMap(n => [n, n * 2]); // [1,2, 2,4, 3,6]
```

Creación y verificación de arrays

```
// Array.isArray(valor) → comprueba si es un array
```

Array.isArray([1, 2, 3]); // true

```
// Array.of(...valores) → crea array con los argumentos
```

Array.of(3); // [3]

Array.of(1, 2, 3); // [1,2,3]

```
// Array.from(iterable, mapFn?) → crea desde iterable o array-like, con mapeo opcional
```

Array.from("hola"); // ["h","o","l","a"]

```
Array.from({ length: 4 }, (_, i) => i * 2); // [0,2,4,6]
```

Patrones útiles (inmutables)

Insertar, reemplazar y eliminar sin mutar

```
const arr = [10, 20, 30, 40];
```

```
// Insertar 99 en la posición 2 (sin mutar)
const insertado = arr.toSpliced(2, 0, 99); // [10,20,99,30,40]
```

```
// Reemplazar el valor en la posición 1 por 77 (sin mutar)
const reemplazado = arr.with(1, 77);       // [10,77,30,40]
```

```
// Eliminar el elemento en la posición 0 (sin mutar)
const sinPrimero = arr.toSpliced(0, 1);    // [20,30,40]
```

Ordenar sin mutar

```
const datos = [3, 1, 2];
```

```
// Orden ascendente sin tocar el original
const asc = datos.toSorted((a, b) => a - b); // [1,2,3]
```

Clonar de forma segura (superficial)

```
const original = [{ id: 1 }, { id: 2 }];
```

```
// Copia superficial con spread
const copia = [...original];        // los objetos internos se comparten
```

Copia superficial vs profunda

```
[...], slice() o Array.from() hacen copias superficiales: si el array contiene objetos, estos
se siguen compartiendo. Para una copia profunda, necesitas una estrategia específica (por
ejemplo, estructurada con structuredClone en entornos que lo soportan).
```

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

La manipulación avanzada de arrays obliga a pensar en mutación y copia. Métodos como push o splice cambian el array original; el spread (...) permite crear una copia superficial y flat() reduce niveles de anidación.

IDEAS CLAVE

- Una copia superficial no clona objetos internos.
- Comparar referencias no es lo mismo que comparar el contenido de dos arrays.
- Cuando la claridad importa, crear un nuevo array puede evitar efectos secundarios difíciles de rastrear.

EJEMPLO NUEVO

```
const original = [{ nombre: "Ana" }];
const copia = [...original];

copia.push({ nombre: "Luis" });

console.log(original.length); // 1
console.log(copia.length);    // 2
```

3.3.1. Clases y objetos (I): fundamentos

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

3.3.1 Clases y objetos (I): fundamentos

En JavaScript moderno (ES6+), las clases proporcionan una sintaxis más clara y cercana a otros lenguajes orientados a objetos. En realidad, siguen basándose en prototipos, pero la sintaxis class nos permite crear y gestionar objetos de manera más sencilla y estructurada.

Creación de clases y objetos

```
// Definición de una clase básica
class Persona {
  // El constructor se ejecuta al crear un objeto con "new"
  constructor(nombre, edad) {
    this._nombre = nombre; //Se suele utilizar el guión bajo para indicar que una propiedad es
privada
    this._edad = edad;
  }
}
```

```
// Crear un objeto de la clase Persona
const persona1 = new Persona("Laura", 25);
console.log(persona1.nombre); // "Laura"
console.log(persona1.edad);   // 25
```

Propiedades y métodos

Las clases pueden tener propiedades (variables dentro de los objetos) y métodos (funciones asociadas a los objetos).

```
class Coche {
  constructor(marca, modelo) {
    this._marca = marca;
    this._modelo = modelo;
  }
```

```
  // Método
  arrancar() {
    console.log(`${this._marca} ${this._modelo} está arrancando...`);
  }
}
```

```
const coche1 = new Coche("Toyota", "Corolla");
```

coche1.arrancar(); // "Toyota Corolla está arrancando..."

El método constructor

El constructor es un método especial que se ejecuta automáticamente al crear un nuevo objeto con new. Se utiliza para inicializar propiedades.

```
class Libro {
  constructor(titulo, autor, paginas) {
    this.titulo = titulo;
    this.autor = autor;
    this.paginas = paginas;
  }
```

```
  descripcion() {
    return `${this.titulo}, escrito por ${this.autor}, tiene ${this.paginas} páginas.`;
  }
}
```

```
const libro1 = new Libro("1984", "George Orwell", 328);
console.log(libro1.descripcion());
// "1984, escrito por George Orwell, tiene 328 páginas."
```

Recuerda

Una clase puede tener solo un constructor.

Si no defines un constructor, JavaScript genera uno por defecto sin parámetros.

Getters y Setters

Los getters y setters permiten controlar cómo se accede y modifica una propiedad de un objeto. Con ellos podemos encapsular la lógica de lectura y escritura sin exponer directamente las propiedades internas.

```
class Rectangulo {
  constructor(ancho, alto) {
    this._ancho = ancho;  // Convención: "_" para propiedades "privadas"
    this._alto = alto;
  }
```

```
  // Getter: devuelve el área
  get area() {
    return this._ancho * this._alto;
  }
```

```
  // Setter: valida antes de cambiar el ancho
  set ancho(valor) {
    if (valor > 0) {
      this._ancho = valor;
    } else {
      console.log("El ancho debe ser mayor que 0");
    }
  }
}
```

```
const rect = new Rectangulo(10, 5);
console.log(rect.area); // 50
```

rect.ancho = 20; // Cambia el ancho

```
console.log(rect.area); // 100
```

rect.ancho = -5; // Muestra advertencia, no cambia el valor

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

Una clase reúne estado y comportamiento relacionados. constructor() inicializa la instancia, this se refiere al objeto actual y los getters permiten exponer propiedades calculadas. Los campos privados con # no pueden accederse directamente desde fuera de la clase.

IDEAS CLAVE

- new crea una instancia de la clase.
- El constructor recibe datos iniciales y los guarda normalmente en propiedades de this.
- Los campos #privados forman parte de la sintaxis moderna de JavaScript.

EJEMPLO NUEVO

```
class Cuenta {
  #saldo = 0;

  ingresar(cantidad) {
    this.#saldo += cantidad;
  }

  get saldo() {
    return this.#saldo;
  }
}

const cuenta = new Cuenta();
cuenta.ingresar(50);

console.log(cuenta.saldo); // 50
```

3.3.2. Clases y objetos (II): herencia y encapsulación

Primero aparece tu material original. Después, una ampliación separada.

CONTENIDO ORIGINAL - SIN MODIFICAR

3.3.2 Clases y objetos (II): herencia y encapsulación

En la programación orientada a objetos, herencia y encapsulación son dos conceptos clave que permiten crear código más organizado, reutilizable y seguro. En JavaScript se implementan a través de las clases, sus constructores y los modificadores de acceso.

Herencia

La herencia permite que una clase (subclase) extienda otra clase (superclase), heredando sus propiedades y métodos. Esto evita repetir código y facilita la especialización de clases.

Sintaxis básica con extends

```
// Clase base
class Animal {
  constructor(nombre) {
    this.nombre = nombre;
  }
```

```
  hablar() {
    console.log(`${this.nombre} hace un sonido.`);
  }
}
```

```
// Subclase que hereda de Animal
class Perro extends Animal {
  hablar() {
    console.log(`${this.nombre} ladra: ¡guau guau!`);
  }
}
```

```
const perro = new Perro("Toby");
```

perro.hablar(); // Toby ladra: ¡guau guau!

En este ejemplo, Perro hereda de Animal y sobrescribe (override) el método hablar.

Uso de super

La palabra clave super permite llamar al constructor o métodos de la clase padre.

```
class Gato extends Animal {
  constructor(nombre, color) {
```

super(nombre); // Llama al constructor de Animal

```
    this.color = color;
  }
```

```
  hablar() {
```

super.hablar(); // Llama al método de la superclase

```
    console.log(`${this.nombre} maúlla: ¡miau!`);
  }
}
```

```
const gato = new Gato("Michi", "negro");
gato.hablar();
// Michi hace un sonido.
// Michi maúlla: ¡miau!
```

Encapsulación

La encapsulación significa proteger el acceso a los datos internos de un objeto, para que solo se manipulen a través de métodos seguros.

En JavaScript moderno se usa el prefijo # para declarar propiedades privadas dentro de una clase.

```
class CuentaBancaria {
```

#saldo; // Propiedad privada

```
  constructor(saldoInicial) {
    this.#saldo = saldoInicial;
  }
```

```
  // Método público para consultar
  verSaldo() {
    return this.#saldo;
  }
```

```
  // Método público para modificar con seguridad
  depositar(cantidad) {
    if (cantidad > 0) {
      this.#saldo += cantidad;
    }
  }
```

```
  retirar(cantidad) {
    if (cantidad > 0 && cantidad <= this.#saldo) {
      this.#saldo -= cantidad;
    } else {
      console.log("Fondos insuficientes o cantidad inválida.");
    }
  }
}
```

```
const cuenta = new CuentaBancaria(100);
console.log(cuenta.verSaldo()); // 100
cuenta.depositar(50);
console.log(cuenta.verSaldo()); // 150
```

cuenta.retirar(200); // Fondos insuficientes

AMPLIACIÓN DIDÁCTICA

Qué significa y cómo leerlo

La herencia permite crear una clase a partir de otra. extends establece la relación y super() permite ejecutar el constructor de la clase padre. La encapsulación consiste en controlar qué estado y comportamiento queda expuesto hacia fuera.

IDEAS CLAVE

- Una subclase hereda métodos de la clase padre.
- super(...) debe usarse correctamente en el constructor de una subclase antes de usar this.
- Los campos privados # de una clase no pasan a ser accesibles directamente desde una subclase.

EJEMPLO NUEVO

```
class Animal {
  constructor(nombre) {
    this.nombre = nombre;
  }

  hablar() {
    return "...";
  }
}

class Perro extends Animal {
  hablar() {
    return "Guau";
  }
}

const perro = new Perro("Toby");
console.log(perro.hablar());
```

REPASO FINAL Y MAPA MENTAL

Condicionales decidir entre caminos.
Bucles repetir trabajo de forma controlada.
Objetos y colecciones representar y manejar datos (String, Number, Math, Date, Set, Map, Symbol).
Funciones organizar lógica, reutilizarla y trabajar con callbacks y closures.
Arrays almacenar colecciones y transformarlas, buscarlas y ordenarlas.
Clases agrupar estado y comportamiento, con herencia y encapsulación.

Orden recomendado para estudiar

• 1. Domina if, else, switch y operadores lógicos.

• 2. Practica for, while, do...while y entiende bien break/continue.

• 3. Aprende String, Number, Math, Date, Boolean, Set, Map y Symbol con ejemplos pequeños.

• 4. Haz funciones simples antes de entrar en callbacks y closures.

• 5. Dedica bastante práctica a map, filter, reduce, búsqueda y ordenación de arrays.

• 6. Termina con clases, constructor, this, getters, campos privados, extends y super.

La idea es que puedas distinguir en todo momento qué era un apunte de tus archivos y qué parte se ha añadido únicamente para ayudarte a entenderlo.